Lab Task 1: Python Classes and Memory Concepts

1. Instantiate a class & display namespace;

```
class MyClass:
    def __init__(self):
        self.x = 10
        self.y = 20

obj = MyClass()


print(obj.__dict__)
```

```
{'x': 10, 'y': 20}
```

2. Function student_data()

```
def student_data(student_id, name=None, student_class=None):
    print("Student ID:", student_id)

    if name:
        print("Name:", name)
    if student_class:
        print("Class:", student_class)

# Calls
student_data(101)
student_data(102, "Ali", "10th")
```

```
Student ID: 101
Student ID: 102
Name: Ali
Class: 10th
```

3. Class type, **dict**, and **module**

```
class Student:
    pass

print(type(Student))
print(Student.__dict__.keys())
print(Student.__module__)
```

```
<class 'type'>
dict_keys(['__module__', '__dict__', '__weakref__', '__doc__'])
__main__
```

4. Two empty classes check

```
class Student:
    pass

class Marks:
    pass

s = Student()
m = Marks()

# Check instances
print(isinstance(s, Student))
print(isinstance(m, Marks))

# Check inheritance from object
print(issubclass(Student, object))
print(issubclass(Marks, object))
```

```
True
True
True
True
```

5. Modify attributes

```
class Student:
    def __init__(self, name, marks):
        self.name = name
        self.marks = marks

s = Student("Nida", 99)

print("Original:", s.name, s.marks)


s.name = "Hani"
s.marks = 99

print("Updated:", s.name, s.marks)
```

```
Original: Nida 99
Updated: Hani 99
```

6. Add and delete attributes

```
class Student:
    def __init__(self, sid, name):
        self.student_id = sid
        self.student_name = name

s = Student(1, "Ali")


s.student_class = "10th"

print("Before deletion:", s.__dict__)

del s.student_name

print("After deletion:", s.__dict__)
```

```
Before deletion: {'student_id': 1, 'student_name': 'Ali', 'student_class': '10th'}
After deletion: {'student_id': 1, 'student_class': '10th'}
```

7. Display all attributes method

```
class Student:
    def __init__(self, sid, name, sclass):
        self.student_id = sid
        self.student_name = name
        self.student_class = sclass

    def display(self):
        for key, value in self.__dict__.items():
            print(key, ":", value)

s = Student(1, "Ali", "10th")
s.display()
```

```
student_id : 1
student_name : Ali
student_class : 10th
```

8. Two instances with structured output

```
class Student:
    def __init__(self, sid, name, marks):
        self.sid = sid
        self.name = name
        self.marks = marks

    def display(self):
        print(f"ID: {self.sid}, Name: {self.name}, Marks: {self.marks}")

student1 = Student(1, "Nida", 95)
student2 = Student(2, "Hani", 95)

student1.display()
student2.display()
```

```
ID: 1, Name: Nida, Marks: 95
ID: 2, Name: Hani, Marks: 95
```

9. Memory allocation analysis

```
import sys

class Student:
    def __init__(self, name):
        self.name = name

s = Student("Ali")

print("Size of object:", sys.getsizeof(s))
print("Size of dict:", sys.getsizeof(s.__dict__))
```

```
Size of object: 48
Size of dict: 296
```

10. Reduce memory usage (**slots**)

```
class Student:
    __slots__ = ['name']

    def __init__(self, name):
        self.name = name

s = Student("Nida")

import sys
print("Reduced size:", sys.getsizeof(s))
```

```
Reduced size: 40
```

Lab Task 2: Object-Oriented Design and Area Comparison

1. Classes for Trapezoid and Parallelogram

```
class Trapezoid:
    def __init__(self, a, b, height):
        self.a = a
        self.b = b
        self.height = height

    def area(self):
        return 0.5 * (self.a + self.b) * self.height


class Parallelogram:
    def __init__(self, base, height):
        self.base = base
        self.height = height

    def area(self):
        return self.base * self.height



t = Trapezoid(4, 6, 5)
p = Parallelogram(5, 4)

print("Trapezoid Area:", t.area())
print("Parallelogram Area:", p.area())
```

```
Trapezoid Area: 25.0
Parallelogram Area: 20
```

2. Comparison Mechanism for Areas

```
t = Trapezoid(4, 6, 5)
p = Parallelogram(5, 4)

if t.area() > p.area():
    print("Trapezoid has larger area")
elif t.area() < p.area():
    print("Parallelogram has larger area")
else:
    print("Both have equal area")
```

```
Trapezoid has larger area
```

3. Store Areas in Dictionary

```
areas = {}

t = Trapezoid(4, 6, 5)
p = Parallelogram(5, 4)

areas["Trapezoid"] = t.area()
areas["Parallelogram"] = p.area()

print(areas)
```

```
{'Trapezoid': 25.0, 'Parallelogram': 20}
```

4. Iterate Over Multiple Dimensions

```
trapezoids = [(4, 6, 5), (3, 5, 4), (6, 8, 7)]
parallelograms = [(5, 4), (6, 3), (7, 6)]

for a, b, h in trapezoids:
    t = Trapezoid(a, b, h)
    print("Trapezoid Area:", t.area())

for b, h in parallelograms:
    p = Parallelogram(b, h)
    print("Parallelogram Area:", p.area())
```

```
Trapezoid Area: 25.0
Trapezoid Area: 16.0
Trapezoid Area: 49.0
Parallelogram Area: 20
Parallelogram Area: 18
Parallelogram Area: 42
```

5. Find Shape with Maximum Area Using Dictionary

```
areas = {}

trapezoids = [(4, 6, 5), (3, 5, 4)]
parallelograms = [(5, 4), (7, 6)]


for i, (a, b, h) in enumerate(trapezoids, 1):
    t = Trapezoid(a, b, h)
    areas[f"Trapezoid_{i}"] = t.area()

for i, (b, h) in enumerate(parallelograms, 1):
    p = Parallelogram(b, h)
    areas[f"Parallelogram_{i}"] = p.area()


max_shape = max(areas, key=areas.get)

print("All Areas:", areas)
print("Maximum Area Shape:", max_shape)
print("Area:", areas[max_shape])
```

```
All Areas: {'Trapezoid_1': 25.0, 'Trapezoid_2': 16.0, 'Parallelogram_1': 20, 'Parallelogram_2': 42}
Maximum Area Shape: Parallelogram_2
Area: 42
```